

Python Decorators – Complete Documentation

Training Agenda

1. Real-world need for decorators
2. Functions as objects
3. Function reference and del
4. Nested functions
5. Functions as arguments
6. Returning functions from functions
7. First-class functions
8. What is a decorator?
9. Basic decorator syntax
10. Decorator execution flow

@functools.wraps

Decorators with *args and **kwargs

11. Parameterized decorators
12. Multiple decorators
13. Real-world E-Commerce use cases
14. Decorator best practices
15. Practice assignment
16. Interview questions

Real-World Problems – Why Do We Need Python Decorators?

Problem 1 – Repeated Authentication/Authorization Logic

Imagine an E-commerce application with multiple operations:

- `view_orders()`
- `cancel_order()`
- `update_product()`
- `delete_product()`
- `view_sales_report()`

Some of these operations require the user to be authenticated or have a specific role.

Without a reusable mechanism, we may end up writing the same authorization logic inside multiple functions.

```
def delete_product(user):  
    if user["role"] != "admin":  
        print("Access Denied")  
        return  
    print("Product deleted")
```

```
def update_product(user):  
    if user["role"] != "admin":  
        print("Access Denied")  
        return  
    print("Product updated")
```

Problem

The same authentication/authorization logic is repeated across multiple functions.

As the application grows:

- Code duplication increases
- Changes become difficult
- Maintaining common logic becomes harder

Question:

Can we write the authorization logic once and apply it to multiple functions?

Problem 2 – Repeated Logging and Monitoring Logic

Suppose we want to log important E-commerce operations such as:

- Customer login
- Adding a product to cart
- Placing an order
- Cancelling an order
- Updating product information

We may write logging code inside every function:

```
def place_order():  
    print("place_order started")
```

```
# Order processing logic
```

```
print("place_order completed")
```

```
def cancel_order():  
    print("cancel_order started")
```

```
# Order cancellation logic
```

```
print("cancel_order completed")
```

Problem

The actual business logic becomes mixed with logging logic.

If we need to add logging to 20 functions, we may have to modify all 20 functions.

Question:

Can we keep the business logic separate and add logging wherever required?

Without decorators, we may repeat the same code inside many functions. This makes the application harder to maintain.

Question: Can we add common behavior to many functions without changing the business logic inside each function?

Yes. Python decorators provide a clean way to do this.

*** * * ** *****

Why Are Decorators Possible?

Before learning decorators, we need to understand one important Python concept:

In Python, functions are objects.

Because functions are objects, we can:

- Assign a function to a variable
- Pass a function as an argument
- Return a function from another function

These capabilities make decorators possible.

Functions as Objects

In Python, functions are objects. Because of this, a function can be assigned to a variable, passed as an argument, and returned from another function.

```
def my_function():  
    print("I am a function.")  
description = my_function  
description()
```

Important: Use `my_function` without parentheses when assigning the function itself. `my_function()` calls the function.

Function Reference and del

A function can have multiple references. `del` removes a variable/reference; it does not directly delete the function object.

```
def fun1():  
    print("Hello")  
  
fun1()  
  
f2 = fun1  
print(id(fun1))  
print(id(f2))  
  
f2()  
  
del f2  
  
fun1()    # Still works  
  
# f2()    # NameError: f2 is no longer defined
```

Key point: After `del f2`, the name `f2` is removed. `fun1` still refers to the same function object.

Nested Functions

A nested function is a function defined inside another function. It is useful when a helper function should be kept inside an outer function.

```
def outer():  
    print("This is outer function.")  
    def inner():  
        print("This is inner function.")  
    inner()  
  
    return inner  
k = outer()  
k()
```

Function as an Argument

A function can be passed to another function just like any other object.

```
def square(number):
    return number * number
def apply_operation(operation, value):
    return operation(value)
print(apply_operation(square, 5))
```

E-Commerce example: A product-filtering function can be passed to another function to select products based on stock, price, category, or rating.

Returning a Function from a Function

A function can also return another function. The returned function can be stored and called later.

```
def get_order_processor(priority):
    def standard_order():
        return "Processing standard order"
    def priority_order():
        return "Processing priority order"
    if priority:
        return priority_order
    return standard_order
process = get_order_processor(True)
print(process())
```

First-Class Functions

Functions are called first-class objects because they can be used like other Python objects.

- Assigned to variables
- Passed as arguments
- Returned from functions
- Stored in collections

What is a Python Decorator?

Decorator is a function; Decorator is always going to take a function as argument and do some enhancement and return some output function.

This is also called **metaprogramming** as a part of the program tries to modify another part of the program at compile time.

A decorator in Python is a function that modifies the behaviour of another function without changing its actual code.

The main advantage is that we can add reusable behavior without changing the original function's business logic.

Common uses: Logging, authentication, authorization, execution-time monitoring, validation, caching, and auditing.

Basic Decorator

```
def decorator(func):
    def wrapper(*args, **kwargs):
        print("Before function call")
        result = func(*args, **kwargs)
        print("After function call")
        return result
    return wrapper
@decorator
def process_order():
    print("Processing order")
process_order()
```

```
@decorator means: process_order = decorator(process_order)
```

Decorator Execution Flow

When Python reads a decorated function, the original function is passed to the decorator and the returned function replaces the original name.

```
@decorator
def process_order():
    print("Processing order")
# Equivalent to:
process_order = decorator(process_order)
```

When `process_order()` is called, the wrapper runs. The wrapper can execute code before and after calling the original function.

Decorator and Metaprogramming

Decorators are a form of metaprogramming because they allow one part of a program to modify or wrap the behavior of another part.

Important: In normal Python decorator usage, this wrapping happens at runtime. It should not be described as modifying the program at compile time.

Decorators with `*args` and `**kwargs`

Using `*args` and `**kwargs` makes a decorator reusable for functions with different positional and keyword arguments.

```

from functools import wraps
def log_call(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}')
        result = func(*args, **kwargs)
        print(f'{func.__name__} completed')
        return result
    return wrapper
@log_call
def add_to_cart(product_id, quantity):
    print(f'Product {product_id} added: {quantity}')
add_to_cart(101, 2)

```

@functools.wraps

@functools.wraps(func) preserves important metadata of the original function, such as its name and docstring.

```

from functools import wraps
def my_decorator(func):
    @wraps(func)
    def wrapper():
        print("Before function call")
        return func()
    return wrapper
@my_decorator
def greet():

```

```

    """This function says hello."""

```

```

    print("Hello")
    print(greet.__name__)
    print(greet.__doc__)

```

Without @wraps: The decorated function may appear to have the name and docstring of wrapper.

With @wraps: The original function's metadata is preserved.

Parameterized Decorators

Sometimes we need to pass a value to the decorator itself. This requires one extra function level.

```

from functools import wraps
current_user = {
    "username": "shop_admin",
    "role": "admin"
}

```

```
def require_role(required_role):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            if current_user["role"] != required_role:
                raise PermissionError("Unauthorized")
            return func(*args, **kwargs)
        return wrapper
    return decorator
@require_role("admin")
def delete_product():
    print("Product deleted")
delete_product()
```

Flow: require_role("admin") → returns decorator → decorator receives delete_product → returns wrapper.

Real-World Use Case 1: Authentication / Authorization

An E-Commerce application may allow only authorized users or roles to perform sensitive operations.

```
from functools import wraps
allowed_roles = {"admin", "manager"}
def authenticate(func):
    @wraps(func)
    def wrapper(role, *args, **kwargs):
        if role in allowed_roles:
            print("Access Granted")
            return func(role, *args, **kwargs)
        print("Access Denied")
    return wrapper
@authenticate
def dashboard(role):
    print(f'Welcome {role} to dashboard')
dashboard("admin")
dashboard("customer")
```

Real-World Use Case 2: Execution Time Monitoring

We can measure how long a function takes without adding timing code to every business function.

```

import time
from functools import wraps
def track_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f'{func.__name__} took {end - start:.2f} seconds')
        return result
    return wrapper
@track_time
def generate_report():
    time.sleep(2)
    print("Generating report...")
generate_report()

```

Why perf_counter(): It is designed for measuring elapsed time and is suitable for execution-time measurement.

Real-World Use Case 3: Logging Activity

A logging decorator can record when an important E-Commerce operation starts and completes.

```

import logging
from functools import wraps
logging.basicConfig(
    level=logging.INFO,
    format="%asctime)s - %(levelname)s - %(message)s"
)
def log_activity(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        logging.info(f'Function '{func.__name__}' started')
        result = func(*args, **kwargs)
        logging.info(f'Function '{func.__name__}' completed')
        return result
    return wrapper

```

Possible E-Commerce functions: login(), add_to_cart(), place_order(), cancel_order(), and update_product().

Real-World Use Case 4: Validation

A decorator can perform common validation before allowing a business function to execute.

```

from functools import wraps
def require_positive_quantity(func):
    @wraps(func)
    def wrapper(quantity, *args, **kwargs):
        if quantity <= 0:
            raise ValueError("Quantity must be greater than zero.")
        return func(quantity, *args, **kwargs)
    return wrapper
@require_positive_quantity
def add_product_to_cart(quantity):
    print(f'{quantity} product(s) added to cart')
add_product_to_cart(2)

```

Benefit: The validation rule can be reused by multiple functions that accept a quantity.

Multiple Decorators

More than one decorator can be applied to the same function.

```

@log_activity
@track_time
@authenticate
def process_order():
    print("Processing order")

```

Internal structure:

```

process_order = log_activity(
    track_time(
        authenticate(process_order)
    )
)

```

Remember: Decorator application is from bottom to top. The decorator closest to the function is applied first.

Decorator vs Normal Function

- Normal function: performs a specific business operation.
- Decorator: adds reusable behavior around another function.
- Business logic remains inside the original function.
- Common cross-cutting concerns can be kept in decorators.

Best Practices

- Use `@functools.wraps` in function decorators.
- Use `*args` and `**kwargs` when the decorator should support different function signatures.
- Keep decorators focused on one responsibility.
- Use meaningful decorator names such as `@authenticate`, `@track_time`, or `@log_activity`.
- Avoid putting core business logic inside a generic decorator.
- Keep exception handling and logging appropriate for the application's needs.

Common Mistakes

- Using `func()` instead of `func` when passing a function as an argument.
- Forgetting to return the result from `wrapper()`.
- Forgetting `@wraps(func)`, which can hide function metadata.
- Writing a wrapper that accepts only fixed arguments when the decorator should be reusable.
- Confusing decorator application order when multiple decorators are used.

Practice Assignment

Create decorators for the following E-Commerce scenarios:

17. **Login validation** – Create a decorator that checks whether a customer is logged in before checkout.
18. **Admin authorization** – Allow only an admin to delete or update a product.
19. **Activity logging** – Log when `add_to_cart()` and `remove_from_cart()` start and complete.
20. **Execution time** – Measure the execution time of `search_product()`.
21. **Quantity validation** – Prevent `add_to_cart()` from accepting zero or negative quantities.

Final Summary

- Functions are objects in Python.
- Functions can be assigned, passed, and returned.
- Nested functions are useful for helper logic and decorators.
- A decorator adds reusable behavior around another function.

`@functools.wraps` preserves the original function's metadata.

- `*args` and `**kwargs` make decorators more reusable.
- Parameterized decorators allow configurable behavior.
- Multiple decorators are applied from bottom to top.
- Decorators are especially useful for cross-cutting concerns such as authentication, logging, timing, and validation.